

Support automation that knows when to answer and when to hand over.

*A retrieval-augmented answering system for CloudServe Solutions,
built to move avoidable tickets off the queue without sending
confident answers to questions the docs cannot answer.*

AUTHOR	Alok Kulkarni
PROJECT	Forward Deployed AI Engineering Capstone
CLIENT	CloudServe Solutions (fictional)
SUBMISSION	20 September 2026
ARTEFACT	02_Report / AlokKulkarni_Capstone_Report.pdf
CODE	04_Source_Code / commit a288388
EVIDENCE	run_20260920 (gpt-4o-mini + gpt-4o, 80 / 80, graded)
FREE-TIER	b21_local_80_20260919 (llama3.1-8b + qwen2.5-7b, 80 / 80)

Contents

1.	Executive summary	3
2.	The problem	4
3.	Discovery findings	6
4.	Requirements	8
5.	Architecture and design	10
6.	Implementation	13
7.	Evaluation	15
8.	Governance and risk	18
9.	The requirements revision	21
10.	Conclusions	23
A.	Appendix A — Evaluation results table	25
B.	Appendix B — Fairness audit table	27
C.	Appendix C — Decision index (D-01 through D-16)	28
D.	Appendix D — Declaration and AI-tool use	29

SECTION 1

Executive summary

CloudServe Solutions asked for a chatbot. The problem behind the request was that a support queue of five hundred tickets a week had grown faster than headcount, and a large share of those tickets were questions their own documentation already answered. This project built a support-answering system that reads a ticket, searches the docs, drafts a reply, checks it against six safety guardrails, and either sends it, blocks it, or hands it to a human with the passages and draft already attached, six components, one after the other, with the routing decision made last from the results of the five before it.

The graded configuration: `gpt-4o-mini` for drafting, `gpt-4o` for guardrails, guardrails on, confidence threshold 0.85, model-response cache on, run 20 September 2026. The system processed all 80 validation tickets in a single unattended run and produced a metrics report and a reconciling decision log. 60 per cent of tickets were auto-answered and 40 per cent held for a human; of those held, 24 were escalated by the router and 8 blocked by a guardrail. No private-data leaks reached an outbound reply. Retrieval hit@3 was 98.1 per cent. Intent accuracy 88.8 per cent. Decisions logged 80 of 80 and reconciled clean. Wall clock 6.8 minutes, cost \$0.80. The same evaluation on the free-tier local stack (`llama3.1-8b-ctx8k` + `qwen2.5-7b-ctx8k`, 19 September) also completed 80 of 80, at \$0.00 and 67 minutes, and is retained as evidence that the free-model gate the Build Spec rewards is met (D-01b).

The headline resolution figure hides two failure modes with opposite fixes, and that split is the caveat the evaluation section expands on. 13 of the 48 sent replies went to tickets the labels say should have been held, and 13 tickets that could have been answered were held. First-contact resolution measured as agreement with the labels is therefore 43.8 per cent, the human baseline was 42, so this is a statistical tie, not a lift. Escalation rate is 40 per cent against a 30 per cent target. Latency p95 is 6.7 seconds against a 3-second one, a miss by rather less than the earlier local run implied, the guardrail stage sits at 2.3 seconds p50 rather than the 30-plus of the local configuration, and D-12's concurrency plus the tightened D-11 pattern set do most of the work.

The system does what CloudServe needs, it moves tickets off the queue safely, and every ticket it holds reaches a human with the passages, top-three intent alternatives and any draft attached, but it does not yet resolve them more correctly than the humans were. Closing that gap needs the feedback loop the architecture doc designs and does not build, plus a query-rewriting pass at retrieval time the fairness audit points at. Both are named in Section 10 as the next two pieces of work.

SECTION 2

The problem

What CloudServe asked for

CloudServe Solutions is a mid-sized cloud infrastructure vendor, about 150 employees, 200-odd corporate customers, roughly \$12M in annual revenue. Support is the part of their operation that has broken first. The head of support, Marcus Adeyemi, took the meeting with a list of numbers. His team of six is receiving upwards of five hundred tickets a week; average time to first reply is between eight and twelve hours against a two-hour SLA; first-contact resolution sits at 42 per cent; customer satisfaction has fallen to 3.2 out of 5; two experienced agents have left in the last quarter and cited workload.

The request that came out of that meeting was for a chatbot. The word carried an assumption about the shape of the solution, a conversational front-end that would take the initial contact off the humans. Building exactly that would have satisfied the request and failed the problem, because a chatbot is a delivery mechanism, not an answer to the question of where the answers come from, whether they are correct, and what happens when they are not known.

What discovery revealed

The Stage 1 workbook and the five stakeholder interview transcripts, read alongside the 500 development tickets, reshape the picture. Three findings, evidenced below in Section 3, did the reshaping.

First, a substantial share of the incoming volume is already answered somewhere in CloudServe's own documentation. Ines Varga, the technical writer, put the figure at about seven in ten. Nobody at CloudServe had written that number down before discovery, and finding it changed the shape of what the system had to be. The lever is answerability against the corpus, not conversation.

Second, the tickets that already have a documented answer are being escalated as if they do not, and the tickets that genuinely need a human are being buried underneath them. Sofia Restrepo, the tier-one agent who sees every ticket before anyone else, described her working day as a triage problem, not a knowledge problem. The queue is a sorting failure more than a capacity failure.

Third, escalations that do reach tier two arrive stripped of the work the earlier stages already did. Daniel Okonkwo asked for one specific thing: when a ticket is handed to him, give him what the system already searched, what it was uncertain about, and any draft it had produced. That request became the EscalationBundle contract in FR-11 and it is why the router runs after generation and the guardrails rather than before them.

The gap between the request and the real problem

The distance between what was asked for and what discovery pointed at is the distance between a delivery mechanism and an outcome. The chatbot framing would have produced a system that engaged customers conversationally and still passed most of what it received to a human, because a chat interface does not by itself resolve anything. The framing this project used instead, a router that reads, searches, drafts, checks, and only then decides, measures success by what leaves the queue correctly and what reaches a human with useful context attached. The mechanism became a background implementation detail; the FastAPI endpoint that exposes the pipeline is a demo affordance and a Prometheus scrape target, and no requirement asks for it.

The tension between the two framings never fully resolves. Marcus, the person who signed off the work, still tends to describe the outcome in chatbot language, because a chatbot is the thing he could picture. Ines and

Daniel describe the outcome in queue and context language, because that is the thing they work in. The PRD reconciles the two by taking Marcus's numeric targets as tier-one measures, Ines's answerability observation as a non-functional constraint on retrieval, and Daniel's context request as the FR-11 EscalationBundle contract. The revision log shows where that reconciliation was rewritten mid-build.

SECTION 3

Discovery findings

Discovery drew from two sources: the five stakeholder interview transcripts and the 500 development tickets in `development_tickets.json`. The transcripts were read once for what people said, once for what they contradicted each other on, and once for what they seemed not to know. The ticket data was used to settle the three contradictions and to test each finding against real volume. The Stage 1 workbook holds the full tables; this section presents the four findings that changed the design and cites the specific evidence for each.

Finding one: a documented answer already exists for most tickets

Ines Varga, in interview four (EV-I1), estimated that seven of every ten incoming tickets had an existing article that answered them. The 500 development tickets, scored against the 29-article corpus, agree: 71.4 per cent are marked `answerable_from_docs = true`. This is the single largest lever in the system. Retrieval quality (hit rate at $k=3$ and the answerability of what comes back) matters more than any other technical measure. The confidence threshold has to be tuned to the retrieval score, not to the classifier score alone.

The finding also constrains what a good outcome looks like. If seven in ten tickets are answerable, an escalation rate above 30 per cent means the system is holding tickets it could have answered. The 40 per cent measured on the graded run is a symptom of the confidence-floor and guardrail-blocking discussion in Section 7, not of the corpus.

Finding two: Marcus wants nothing wrong more than he wants everything answered

Marcus Adeyemi, in interview one (EV-M3), was explicit about the trade-off. His exact words were: *"I would rather it said nothing than said something wrong."* This is precision over recall, stated as a management preference and repeated in different words twice in the same conversation. It is the single evidence tag that does the most design work in the PRD.

It is why the confidence floor exists (FR-19), why the D-07 policy list escalates four intent categories at any confidence, and why every one of the six guardrails blocks rather than warns. It is also why the D-05b confidence-threshold sweep was set up to look for a threshold satisfying the 0.95 precision floor, none did, and that finding is honestly reported in the ADR and reappears in Section 7.

Finding three: escalations arrive without context

Daniel Okonkwo, in interview three (EV-D1, EV-D3), described the current state of an escalated ticket as "a raw ticket with a note attached". Tier one has already searched the docs, formed a hypothesis about the intent, and sometimes drafted a reply that the customer never sees. None of that reaches him. His request was concrete: when you hand me a ticket, give me what you already searched, what you thought it was, and any draft you had. The system must show its work.

This became FR-11, every escalation carries an `EscalationBundle`: the retrieved passages, the classifier's top-three alternatives, the draft (even a draft the guardrails blocked, flagged as `draft_blocked`), and the point at which the system was unsure. It is also why `route()` runs after generation and validation rather than before them. Routing earlier, the pattern the first draft of the architecture used, leaves the bundle empty, because the passages, draft and guardrail verdicts do not exist yet.

Finding four: the ticket data disagrees with itself

The 500 development tickets and the 80 validation tickets are labelled at the ticket level. A label audit on 20 September (documented in `docs/architecture.md` under Dataset labels) found fifty groups of identical subject and body that disagree on `expected_route` or `answerable_from_docs`, 144 tickets in total. Two of these pairs sit inside the validation set itself (VAL-0012/VAL-0034 and VAL-0060/VAL-0061), which means no system can be right on both members of either pair. Twenty-one of the eighty validation tickets carry contested text; 10 of the 26 wrong decisions on the 20 September graded run land on them.

This is not a finding about the system, it is a finding about the measurement. It is reported here because it sets an upper bound on what any score against the validation set can mean, and because the labels were not corrected. Editing ground truth to match model output would make every figure unfalsifiable, and the hidden evaluation set carries whatever labels it carries. The alternative, reporting the honest figure with the noise floor attached, is what Section 7 does.

What discovery ruled out

Two candidate features were ruled out during discovery rather than during build. A holding-reply feature, automatic acknowledgement of receipt while a human works on the real reply, was considered because the SLA breach is on first response. It was ruled out because Sofia's interview (EV-S2) described the current automated acknowledgement as "the thing customers already ignore". The PRD explicitly places it out of scope (Table 6). A conversational multi-turn interface was ruled out for the same reason as the chatbot framing: the discovery evidence points at answerability and triage, not at conversational back-and-forth.

SECTION 4

Requirements

The PRD (workbooks/Stage_2_PRD_Template.docx) carries 22 functional requirements and 8 non-functional requirements. Every one of them names, in a Traceability column, the piece of discovery evidence that produced it and the acceptance criterion it maps to. The full document is in the workbooks folder; this section states the shape of the requirements set and highlights the ones a reader of the report needs to be able to trace to a finding.

How requirements followed from findings

The four discovery findings map onto the requirement set as follows. Finding one, most tickets are answerable from the docs, produced the retrieval requirements (FR-06 through FR-08), the D-02, D-02a, D-04 and D-09 decisions on embedding model, distance metric, chunking and query text, and NFR-01 on retrieval hit@3. Finding two, precision preferred over recall, produced FR-19 (confidence floor), FR-10 with D-07 (never auto-respond intents), and NFR-02 (hallucination). Finding three, escalations arrive without context, produced FR-11 (EscalationBundle) and the architectural decision to place `route()` last in the pipeline. Finding four, labels disagree with themselves, produced NFR-08 on evaluation honesty and the discipline in `results_table.py` that carries a limitations block with every metrics table.

The requirements at a glance

The table below is not the PRD, it is an orientation to where things live so a reader can trace forward from a finding or backward from a line of code.

Component / concern	Functional	Non-functional	Anchoring decision
Ingest	FR-01, FR-02, FR-03	NFR-06 (availability)	,
Classify	FR-04, FR-05	NFR-04 (fairness)	PR-CLASSIFY-01 (what the classifier does not see)
Retrieve	FR-06, FR-07, FR-08	NFR-01 (hit@3)	D-02, D-02a, D-04, D-09
Generate	FR-13, FR-14, FR-15	NFR-02 (hallucination)	D-06 amended by D-06a
Validate (six guardrails)	FR-15 – FR-19	NFR-03 (PII zero)	D-11, D-12, D-14, D-16
Route	FR-09 – FR-12	NFR-05 (variation)	D-05b, D-07, D-14, D-15
Decision log	FR-20, FR-21	NFR-07 (auditability)	D-03, D-03a, D-03b
Metrics	FR-22	NFR-06	,
Kill switch	,	R-08 mitigation	D-15

Table 4.1 — Where each component's requirements live, and the ADR that carries the anchoring design decision.

Two requirements that changed shape mid-build

FR-10 v1 said *"tickets marked must_not_auto_respond in the labels must escalate."* This survived contact with the code for exactly one day. Labels are evaluation ground truth and FR-01 v2 keeps them off the `Ticket` object the router sees. The router cannot read a field that does not exist. FR-10 v2 replaces the label reference with a fixed intent policy list, the four categories `compliance_request`, `security_incident`, `feature_request` and `unclear_request`, plus `unknown` for classifier fallback (D-07). EV-DATA-10 shows the label is a function of intent, so the policy list reproduces it at precision and recall 1.000 on all 580 labelled tickets. The change is in the revision log, Table 2, row 3.

FR-19 v1 said *"the confidence floor blocks a low-confidence reply."* D-14 changes what the floor routes to. A confidence-floor failure reads the classifier's confidence, which was known before a word was drafted, so `block` names the wrong thing, there is no draft to be blocked. FR-19 v2 escalates on low confidence. On the 80-ticket run recorded before that fix, 17 of the 33 blocks had the confidence floor as their only failing check, and 7 of those 17 were D-07 policy tickets rule 3 should have taken. The change moved the floor from rule 2 to rule 6 in the router and is recorded in the revision log, Table 2, row 6.

Traceability, spot-checked

A reader picking a random requirement should be able to follow it to code, to a test, and back to a finding. Two worked examples:

FR-11 (EscalationBundle) → **EV-D1, EV-D3**. `src/route.py::_build_bundle` assembles the bundle when the router returns `escalate`. `tests/test_route.py` asserts the bundle contains passages, top-3 alternatives, draft and reason. The evidence is Daniel Okonkwo's request in interview three, cited by the requirement.

FR-16 (PII guardrail) → **EV-M3, EV-M5, R-02**. `src/guardrails.py::PIIGuardrail` runs on every drafted reply and returns a blocking verdict when a value-shaped PII detection fires (D-11 tightens the pattern set). `tests/test_guardrails_prd.py::test_pii_blocks_email` confirms the block. The evidence is Marcus's precision preference and the compliance-audit condition R-02.

The audit-friendly way to check this at scale is the one the assessor will use: pick any FR ID, `grep -r "FR-11" src/ tests/ prompts/`, and the file paths and line numbers land in one screen.

SECTION 5

Architecture and design

The system is a Router agent. Six components run in sequence: `ingest` → `classify` → `retrieve` → `generate` → `validate` → `route`. The order matters, and one aspect of it is worth stating first because it is not the pattern the term "router" first suggests: the routing decision runs *last*, not somewhere in the middle. Every ticket is classified, searched, drafted and checked, and only then does `route()` read what those four produced and pick one of three outcomes: `auto_respond`, `escalate` or `block`. It has to run last because two of its eight rules read results that do not exist any earlier: whether a blocking guardrail failed, and whether the generator declined to answer.

The first draft of this architecture put `ROUTE` fourth, before `GENERATE`. That order was never what the code did and could not be made to work. routing before validation leaves `block` unreachable and reduces the guardrails to advisory checks. The documented order was corrected on 20 September in the `architecture.md` changelog.

The pipeline

INGEST → CLASSIFY → RETRIEVE → GENERATE → VALIDATE → ROUTE

The decision log and the metrics writer are the two cross-cutting concerns; each stage writes one row to the log before the next stage starts, so at end-of-run reconciliation the tickets processed and the decisions logged agree exactly, scoped by `run_id` (D-03, D-03b).

Figure 5.1 — The six-stage pipeline. Log and metrics touch every stage; feedback loop is designed but not built (Section 8).

What each component does

Ingest (FR-01, FR-02, FR-03). Takes the raw ticket from any of the four channels, email, live chat, docs_comment, community forum. and produces a normalised `Ticket` object the rest of the pipeline can work with. Handles empty subject (every chat ticket has this by design, 155 of 155 in the dev set), unusual characters and empty body without raising.

Classify (FR-04, FR-05). Returns an intent (one of 22 categories), an urgency, a confidence between 0 and 1, and up to two alternatives. Deliberately does not see `customer_tier`, `customer_region`, `customer_name` or `language_fluency`. This is a fairness decision: tier-aware policy lives in the router, not the classifier. On failure the classifier returns `intent='unknown'` and `confidence=0.0` rather than raising (FR-05).

Retrieve (FR-06, FR-07, FR-08). Takes subject + body (D-09), searches 29 help articles chunked at 800 characters with 120-character overlap (D-04), with the article title and category prepended before embedding (B-30). Embeddings from `all-MiniLM-L6-v2` (D-02, verified 31 Aug). Chroma index in cosine space with a 0.25 relevance floor (D-02a, measured 3 Sep). Below the floor the retriever returns an empty list, retrieval prefers to return nothing over something irrelevant, which the Build Spec calls out explicitly.

Generate (FR-13, FR-14, FR-15). Takes the ticket and the retrieved passages, writes a JSON reply with `answer`, `citations[]`, `confidence` and an `unknown` flag. The ticket text is wrapped in `<<TICKET_START>>` / `<<TICKET_END>>` markers, and any instructions inside those markers are ignored (FR-15). A structural self-check on citations runs once (D-06, amended by D-06a): every cited `doc_id` must be one retrieval returned; a real answer carries at least one citation; an "I don't know" carries none. One retry allowed.

Validate, six guardrails (FR-15 through FR-19). All six block, none warn. PII (FR-16) blocks value-shaped detections of emails, API keys, phone numbers, account numbers and other customers' names. D-11 tightened the pattern set after all three PII blocks in the local run were the judge flagging the words "API keys" or an article id. Grounding (FR-17) checks every factual claim against a retrieved passage. Instruction integrity (FR-18) blocks replies where the ticket managed to redirect the system. Tone and scope blocks commitments about refunds, timelines or the roadmap. Answer relevance blocks replies that do not address the question. Confidence floor (FR-19) checks the classifier confidence against the threshold. The six run concurrently, at most four at once (D-12), and the results come back in guardrail order.

Route (FR-09 through FR-12). A pure function of values earlier stages already computed, no clock, no randomness, no model call of its own (this is what makes A5 hold structurally, not by argument). Eight rules, ordered, first match wins: kill switch (D-15), unlogged decision (D-03b), guardrail block (D-14 excludes confidence-floor from this rule), suspected injection (D-16), D-07 policy list, empty retrieval, generator declined, confidence floor. FR-12 requires the reason to name the cause, so precedence is part of the contract. On `escalate` the router returns an `EscalationBundle` (FR-11), passages, top-3 alternatives, draft, and the point of uncertainty.

What was considered and rejected

The design decisions were recorded as ADRs, one file per non-obvious choice, each with the options considered, what was picked, why, and what would make us change our minds. Sixteen ADRs are in `docs/adr/`. Six of the more consequential "options rejected" moments are:

ADR	What was rejected	In favour of	Why
D-01	Groq free tier	OpenRouter (Llama 3.1 8B)	Setup Guide default; single API key for a broader model catalogue; nothing to install on the assessment machine.
D-02	TF-IDF baseline; BAAI/bge-small-en-v1.5	all-MiniLM-L6-v2	B-07 on 357 dev tickets: 91.0 % top-3 hit rate, within 3 pts of TF-IDF and better on non-fluent English (fairness win).
D-03	PostgreSQL, JSONL	SQLite	No service to provision; standard library only; reconciliation is a single SELECT.
D-06	ReAct, Plan-and-Execute, full Self-RAG	Router with an inline structural critic	Q3 pilot showed search is essentially solved on this corpus; engineering value is above retrieval.
D-06a	Semantic in-loop critic	Structural critic + blocking grounding guardrail	The guardrail blocked 3 of 80 drafts on the local run, so a semantic in-loop critic would recover at most 3 tickets for one more model call each.
D-10	Retrieval-feature answerability gate	No gate	Six retrieval features reach AUC 0.64 (dev, cross-validated). The dev-chosen gate would have held 0 of the local run's 14 wrong sends.

Table 5.1 — Six illustrative ADRs. Appendix C indexes the full sixteen.

Layers

Three layers under the six components. **Application** is the evaluation harness command-line tool (`evaluation/harness.py`) and the FastAPI service (`src/api.py`: POST `/ticket`, GET

`/healthz`, `GET /metrics`, `GET /`). The harness is the graded path. `metrics_report.json` is the artefact A9 and A10 ask for, and `POST /ticket` composes the same six calls in the same order. **Domain** is the six components (`src/ingest.py`, `classify.py`, `retrieve.py`, `route.py`, `generate.py`, `guardrails.py`). **Persistence** is the SQLite decision log at `storage/decisions.db`, the Chroma vector store and the Prometheus metrics endpoint.

The separation is what makes the system testable. Swapping the model provider or the vector store does not touch the domain code, which is why the D-01a transition from a failing free model to paid diagnostics and back to a working free-tier local stack is a config change rather than a rewrite.

SECTION 6

Implementation

How it was built

The code sits in eleven files under `src/`: the six component modules named above, plus `config.py` (settings and environment reads), `logging_store.py` (SQLite decision log with `run_id` scoping), `model_call.py` (a bounded wrapper on every provider call, D-13, `MODEL_CALL_DEADLINE_SECONDS`), `model_cache.py` (on-disk response cache keyed by prompt hash, D-08), and `injection.py` (input-side heuristic, D-16). The evaluation harness at `evaluation/harness.py` composes these into one command that takes an input path and an output path, per the Build Spec's A9 requirement.

The build followed the Stage 4 sprint plan and the day-by-day checkpoints in Section 5 of the Build Specification. Week two, day one: ingest working end to end on a ticket from each of the four channels. Day two: retrieval producing traceable passages from the corpus. Day three: classify + route producing decisions with reasons in the log. Day four: generate + guardrails with a first blocking guardrail firing. Day five: the full 80-ticket unattended run producing `metrics_report.json` without human intervention. Week three was spent on the honest evaluation, the ADRs D-08 through D-16 that closed the loop on issues week two surfaced, and the submission artefacts.

Test coverage

Twenty-two test files under `tests/`, runnable with a single documented command (`python -m pytest tests/ -v`, per A12). Coverage divides into four groups. **Component tests**, one file per `src/` module, exercise each stage in isolation. **Integration tests**, `test_harness_smoke.py`, `test_api_smoke.py`, run the pipeline end to end with stubbed providers. **Guardrail tests**, `test_guardrails.py`, `test_guardrails_prd.py`, `test_guardrail_draft_review.py`, one explicit case per guardrail's blocking condition, satisfying A7. **Governance tests**, `test_fairness_audit.py`, `test_calibration_sheet.py`, `test_gt_response_check.py`, `test_config_keys.py`, enforce the audit-time checks that would otherwise drift.

The `test_config_keys.py` file is the one worth calling out. It fails the CI run if any file under `src/` or `.github/workflows/` matches the four credential patterns the Submission Guide names. `sk-`, `sk_`, `gsk_`, `hf_`. It is the mechanical version of the promise not to commit keys.

What was difficult

Free-tier rate limits (D-01, D-01a, D-08). The Setup Guide's default free model (`llama-3.1-8b-instruct` via OpenRouter) moved to paid pricing during week two, and both recorded free-tier runs (13 and 17 September) lost most calls to outages or HTTP 429. The Build Spec is explicit that *"rate limits are a design problem"* and encourages caching. The response was D-08, an on-disk response cache keyed by prompt hash, retries lifted from 2 to 5, pacing only after a refusal, plus a local Ollama stack (`llama3.1-8b-ctx8k` for drafts, `qwen2.5-7b-ctx8k` for guardrails) that closes the free-model gate the Build Spec rewards. The 19 September local run of 80/80 is the evidence (D-01b). The 20 September graded run reported in Section 7 uses paid providers (`gpt-4o-mini` + `gpt-4o`) because those are the numbers the reviewer will see in production; the free-tier run is retained so the reviewer can also see that the system works without them.

Long-tail latency (D-13). A 20 September provider run hung over seven minutes on a healthy endpoint and had to be killed. Root cause: `MODEL_TIMEOUT_SECONDS` is an `httpx gap-between-bytes` timeout, which a provider streaming keep-alives resets forever. Fix: caller-side deadline, `MODEL_CALL_DEADLINE_SECONDS` (180 s), wrapping every provider call. Without it, one hung call takes A9 with it.

Guardrail latency (D-12). Six guardrails run in turn take roughly 10 s on hosted models and about 30 s on the local stack. Running them through a thread pool with at most four workers at once cut the guardrail stage from 10.1 s to 3.1 s on hosted models (overall ticket p95 $24.2 \rightarrow 12.2$ s), and about $1.11\times$ on the local judge which needs `OLLAMA_NUM_PARALLEL > 1` to see any benefit. Verdicts on 20 replayed drafts were identical either way, so concurrency is a pure win.

Confidence-floor mis-classification (D-14). Before the fix, a low classifier confidence returned `block` at rule 2 alongside guardrail failures on the draft. That names the wrong thing, the floor reads classifier confidence, not the draft, so there is no draft to block. Fixed by moving the floor to rule 6 in the router and returning `escalate`. On the earlier 80-ticket run, 17 of the 33 blocks were floor-only, and 7 of those were D-07 policy tickets rule 3 should have taken.

What I would restructure

The generator and the guardrails should share their model call. Today the generator writes a full draft, then five guardrails each re-read it. Running guardrails against a partial generator response instead of waiting for the whole one is the single largest latency win available. This is the first item in Section 10.

The decision log should not have a `run_id`-scoped reconciliation. This is a symptom of `storage/decisions.db` being a persistent file that survives from run to run. Wiping the file at the start of each harness invocation would remove the scoping code and the class of bug it exists to prevent. The reason I did not restructure this is that persistent `decisions.db` is useful for the demo, one file to browse the whole history, and A8 reconciliation is trivially satisfied by `WHERE run_id = ?`.

The FastAPI service is thin. It exists because a live demo wanted an endpoint and Prometheus wanted a scrape target. If this system had to serve real traffic, the API would need request queuing, per-tenant rate limits, and a webhook back to the ticketing system on `escalate`. Nothing in the current PRD asks for those, so nothing was built.

SECTION 7

Evaluation

Method

The graded configuration ran on 20 September 2026: 80 validation tickets processed in one unattended harness invocation against `gpt-4o-mini` for drafting and `gpt-4o` for the six guardrails, confidence threshold 0.85, guardrails on, `GUARDRAIL_MAX_WORKERS=4`, model-response cache on (D-08). Run ID `harness-20260920T173736Z-8eee28c5`, code `a288388`. Wall clock 406.7 seconds. Model calls 304, of which 31 came from the cache and 273 were live; prompt tokens 489 124, completion tokens 23 147, cost \$0.80, cost saved by cache \$0.09. Metrics report and results table under `evaluation/results/run_20260920/`.

The 19 September free-tier local run (`llama3.1-8b-ctx8k` for drafts, `qwen2.5-7b-ctx8k` for guardrails, `b21_local_80_20260919`) also completed 80 of 80 tickets, at \$0.00 and 67 minutes wall clock, kept as the D-01b evidence that the free-model gate the Build Spec rewards holds. The free run's routing is the same shape (60/40 auto vs held, 0 PII in outbound); it costs six per-ticket seconds in guardrail latency and one to two per-cent on correctness. Section 6 covers why the graded numbers come from the paid stack.

Runs against the hidden evaluation set: **zero**. The hidden set is held by the assessor and the harness accepts an `--input` path so it can be pointed at the file after submission (Build Spec, Section 4). Everything reported below is against the 80-ticket labelled validation split, not the held-out test set.

Business outcomes

The four business measures, run against the labels:

Measure	Baseline	Target	Achieved	Status
First-contact resolution	42 %	60 %	43.8 % correctly resolved (60.0 % auto-sent; 95 % CI 33–55 %)	NOT MET
Mean time to first reply	8 – 12 hrs	< 5 min	5.3 s (median 5.8 s; 7 replayed from cache)	MET
Satisfaction proxy	3.2 / 5	4.0	Not measured to Framework method (see paragraph below)	NOT MEASURED
Escalation rate	58 %	≤ 30 %	40.0 % (24 routed, 8 blocked; 95 % CI 30–51 %)	NOT MET

Table 7.1 — Tier-one business outcomes on the graded 80-ticket run. Full table in Appendix A.

The FCR figure is the interesting one and it deserves the paragraph the Evaluation Framework asks for. 60 per cent of tickets were answered without a human; 40 per cent went to the queue, a clear directional improvement on the 58 per cent baseline escalation rate. 13 of the 48 sent replies went to tickets the labels say should have been held; correctness against `labels.expected_route` is 43.8 per cent, against a 42 per cent human baseline. That is a tie within noise, not a lift. The system moves work off the queue but does not yet resolve *more* of it correctly than the humans were. What has changed is where the failure shows up: the 13 wrong sends will return as repeat contacts or correction tickets rather than as unresolved queue depth. Ines's feedback-loop verdict vocabulary (`sent_as_drafted`, `edited_then_sent`, `wrong_article`, `system_misread`, `no_article_exists`) exists to measure this in production;

today the measurement is against the labels.

Satisfaction is honestly not measured. The Framework asks for a sample of responses scored against a rubric by human reviewers, with sample size, rubric and scorers reported. This run has no such assessment. The nearest evidence is B-18, where one person scored 50 drafts from the 14 September paid run on the judge's RAG rubric (mean answer relevance 4.86/5), a different rubric, a different run and a single scorer, so not a satisfaction proxy. Reporting the gap openly is what the Framework's honesty callout asks for.

Technical performance

Retrieval hit@3 on the 53 answerable tickets is 98.1 per cent. beat the 93.6 per cent baseline. Recall@1 is 78.3 per cent, recall@3 85.9 per cent, recall@5 89.6 per cent. Retrieval, as Sofia's finding predicted, is essentially solved on this corpus.

Intent classification accuracy is 88.8 per cent overall (71 of 80). 15 of 22 classes reach the 85 per cent per-class target; the target is *per class*, so it is met only if every class does. The weakest are `account_access` (0 %, n=4), `integration_help` (0 %, n=3) and `sso_configuration` (33 %, n=1). 16 classes have fewer than five tickets in the validation set, so per-class figures are indicative. The full per-class table is in Appendix A.

Citation accuracy against the labels: for tickets where the generator produced a citation, the cited article was one the labels expect in 95.3 per cent of cases across 53 drafts (recall 82.1 per cent). This checks the *right source*, not *the sentence is supported by that source*, the Framework asks for the latter. Sentence-level support was not assessed on this run, so the figure is reported as "not measured to method". The grounding guardrail did check sentence-level support on every draft and blocked four.

Hallucination rate is similarly reported as not measured. The Framework asks for at least 50 responses assessed independently by two people, with agreement reported. This project had one human assessor. Nearest evidence: on the 50 drafts from the 14 September paid run, one human found an unsupported claim in 1 of 50; the LLM judge in 5 of 50; the judge failed its own calibration check (pooled Spearman 0.35 against the 0.70 threshold) and its scores are excluded from the reported figures.

Latency p95 on the graded (paid) stack is 6.7 seconds against the 3-second target; median 5.8 seconds; maximum 7.0 seconds. The stage-by-stage split (live tickets, n=71): classification p50 1.5 s, retrieval 0.12 s, generation 1.8 s, guardrails 2.3 s. Model calls per ticket average 3.77 (D-12's concurrency plus a per-ticket set that no longer runs every guardrail sequentially). The equivalent figure on the free-tier local stack is 70.4 s p95, kept for the D-01b comparison but not the number that ships. The miss against 3 s is now a factor of two, not twenty, and the design decision the target really questions is the six-guardrail-per-ticket contract itself, the restructure item in Section 6 that runs guardrails against a partial generator response.

Governance conditions

Four conditions, four verdicts:

Condition	Requirement	Result	Status
Private data in outbound text	Zero occurrences	0 (automated scan of all 48 sent replies)	MET
Quality across customer groups	< 5 pp variation	20 pp (language_fluency: non-fluent vs fluent, n=19; not significant at that n)	NOT MET
Decision logging	Complete coverage	80 of 80 logged; reconciles = True	MET

Confidence calibration	Stated $\approx$ observed within 5 pp	Largest gap –35 pp in the 0.7–0.8 band (n=10); 0.8–0.9 band observed 0.94	NOT MET
------------------------	---------------------------------------	---	---------

Table 7.2 — Tier-three governance conditions. Private data and decision logging are the two hard binaries; both hold.

The calibration gap needs a sentence. The operational band (0.8–0.9, n=53) is observed accurate at 94.3 per cent against a stated 84.4 per cent, a 10 pp under-confidence, which is the milder direction of miss but still outside the 5 pp target. The 0.7–0.8 bucket (n=10) is the largest gap in the other direction: stated 0.75, observed 0.40, a 35 pp overconfidence on a small sample. The Brier score is 0.113, an improvement on prior runs and inside the range the PRD's NFR-05 tolerance treats as usable. The condition is called NOT MET because the Framework asks for 5 pp across every band, and none of the runs on this stack has reached that. Sample sizes below n=5 remain the confounder.

The figures above should be treated with caution because

A single run of 80 tickets: one ticket moves a rate by 1.25 points, and 95 per cent confidence intervals on proportions are wide. Nine of the 80 tickets were partly replayed from the D-08 model cache, so for those nine the run describes the run that filled the cache. Three measures, satisfaction proxy, hallucination rate, citation accuracy, are not measured to the Framework's method and are reported as such rather than substituted. Correctness is agreement with dataset labels, and 21 of 80 validation tickets carry text that appears elsewhere in the labelled data with a different `expected_route` or `answerable_from_docs`. Two such pairs sit inside the validation set itself, where no system can be right on both. Hidden evaluation set runs: zero.

SECTION 8

Governance and risk

The Governance Framework asks for five things: a risk register with real mitigations, a decision log covering every automated decision, a fairness audit across customer groups, an incident procedure someone unfamiliar with the system could follow at two in the morning, and a declaration of what the system must never do. This section is the summary; the full workbook and the fairness audit table are in Appendix B and in docs/adr/.

Risk register

Eight risks. The four with a mitigation that is a design property rather than a promise:

ID	Risk	Mitigation in the design
R-01	The system answers confidently and incorrectly	Three independent checks: confidence floor (FR-19, D-05b), grounding guardrail (FR-17) on every draft, and the D-07 policy list (FR-10) for four intents that never auto-respond.
R-02	Private data appears in an outbound response	PII guardrail on every drafted reply, value-shaped patterns (D-11). Result on the 20 Sep 80-ticket run: 0 detections in sent replies, 0 pre-send blocks, D-11's tightening dropped the three false positives the 19 Sep run had.
R-03	A customer's input is treated as an instruction	Ticket text wrapped in <<TICKET_START>> / <<TICKET_END>> markers (FR-15). Input-side heuristic escalates and records suspected injection (D-16, router rule 2b). Six-attack probe on 20 Sep: none succeeded.
R-04	Some customer groups receive worse answers	Fairness audit run every graded run. The 20 Sep run shows a 20 pp gap on language_fluency non-fluent vs fluent (n=19, not significant at that n), down from the 44 pp of the 15 Sep audit; still above the 5 pp target, and named in Section 10 as the second piece of work.
R-06	The model provider becomes unavailable	D-08 response cache, D-13 caller-side deadline, D-01a's paid fallback and the local Ollama stack. The 19 Sep local run's 80/80 completion under a failing free-tier provider is the evidence this holds.
R-08	The system does something we cannot roll back quickly	D-15 kill switch: KILL_SWITCH=1 read on every routing call, no deploy required, takes effect on the next ticket. Test: test_kill_switch_forces_escalate.

Table 8.1 — The six risks with a design-level mitigation. R-05 (documentation staleness) and R-07 (latency under load) are covered by out-of-scope items in the PRD; both are recorded in the workbook.

Fairness audit

The audit. `evaluation/fairness_audit.py`, run on the 80-ticket set, segments outcomes across language fluency, region, tier, channel, and length. The 15 September run (paid providers) found the following variations larger than 5 percentage points. The table is quoted verbatim from the fairness audit output; $\pm$ is the quality-score gap from the best-performing segment in the group. Full table in Appendix B.

Segment	n	Resolution	Quality	Gap from best (pp)
language_fluency = non_fluent	19	62 %	47 %	+25
language_fluency = fluent	61	75 %	72 %	+0
customer_tier = business	30	62 %	50 %	+29
customer_tier = standard	42	86 %	79 %	+0
channel = docs_comment	16	50 %	56 %	+16
channel = forum	11	83 %	73 %	+0

Table 8.2 — Fairness audit, largest gaps. The non-fluent English gap is the R-04 finding this project reports honestly rather than masks.

The non-fluent English gap is the disparity the Governance Framework's callout warned about. *"systems built on retrieval tend to perform noticeably worse on tickets written in non-fluent English, because the retrieval step depends on the phrasing matching the documentation."* The 20 Sep run's 20-point gap on decision correctness at n=19 is not significant at that sample size, but is a real narrowing from the 44 pp of the 15 Sep audit. It still exceeds the Framework's 5-point target and the PRD's own NFR-05 allowance of 15 points. It is named in Section 10 as the second piece of work; the fix is query rewriting at retrieval time to normalise phrasing towards how the docs speak, before deeper embedding swaps.

Decision logging

SQLite at `storage/decisions.db` (D-03). Schema is the Governance Framework's minimum record plus two extensions: `run_id` for per-run scoping (Bug 2 fix, recorded in revision log Table 3 row 6) and a widened `action_taken` vocabulary that distinguishes per-stage outcomes by the `stage` column (row 7). Every decision writes one row before the reply is sent. Reconciliation at end of run is a single `SELECT`, scoped by `run_id`. On the graded run: 80 tickets processed, 80 decisions logged for routing plus per-stage rows for each; reconciliation clean, A8 pass.

Incident procedure and the kill switch

The incident procedure (workbook, Section 5) covers six steps. detect, contain, assess, notify, remediate, review, with named roles, target durations and a fallback if the primary responder is unavailable. The one detail from it that belongs here is the kill switch, because it is the mechanism the containment step depends on.

D-15 replaces the earlier `CONFIDENCE_THRESHOLD=1.01` trick with a dedicated flag. `KILL_SWITCH=1` is read by `route()` on every routing call as rule 0; when set, the router returns `escalate` with reason `kill_switch_active` for every ticket. No deploy is required; the switch takes effect on the next ticket. Any on-call engineer with access to the environment can operate it. The old trick shared a constant with threshold tuning and read in the log as a misconfigured threshold rather than a deliberate halt, and it was untested. D-15 is tested: `tests/test_route.py::test_kill_switch_forces_escalate`.

The declaration

The Governance Framework asks for four statements. Filled in:

Statement	Position
This system must never	send an unreviewed reply that contains another customer's data, an unsupported factual claim, or a commitment about refunds, timelines, or the roadmap.
The mechanism that enforces that is	the six-guardrail block set. pII, grounding, instruction integrity, tone-and-scope, answer relevance, confidence floor, running on every drafted reply, and the router that turns a block into a hold-for-human rather than a hidden reject.
The most likely way it could still cause harm is	a wrong send on an answerable ticket whose retrieved article is itself out of date. The feedback loop the architecture designs but does not build is the missing measurement.
We would not deploy this without first	running the fairness audit in production with a real hidden set, wiring the feedback-loop verdict vocabulary through the reviewer's UI, and closing the confidence-floor calibration gap in the 0.9–1.0 band.

Table 8.3 — Governance declaration.

SECTION 9

The requirements revision

The Stage 5 revision log (`workbooks/Stage_5_PRD_Revision_Log.docx`) records nine substantive changes to the PRD between v1.0 (end of week one) and v2.0 (20 September). The full table sits in the workbook; the four that reshaped the design, as opposed to tightening wording, are covered here.

The four changes that mattered

Change one: FR-10 stops reading a label the router cannot see. v1 said the router must escalate any ticket marked `must_not_auto_respond`. The router does not receive labels; FR-01 v2 keeps them off the `Ticket` object because labels are evaluation ground truth and using them at inference is data leakage. v2 replaces the label reference with a fixed intent policy list of four categories (D-07). EV-DATA-10 shows the flag is a function of intent, so the substitution reproduces the label at precision and recall 1.000 on all 580 labelled tickets. Trigger: the first code review pass on `route.py`. Agreed by the PRD reviewer.

Change two: FR-19 escalates instead of blocking. v1 said a low classifier confidence blocks the reply. D-14 changes this. The confidence floor reads the classifier score, which was known before any draft existed, so `block` names the wrong outcome, there is no draft to block. v2 has the floor return `escalate` and moved it in the router from rule 2 (guardrail block) to rule 6 (after the D-07 policy and the empty-retrieval check). Trigger: an audit of the pre-fix 80-ticket run that showed 17 of 33 blocks were floor-only, 7 of them D-07 policy tickets rule 3 should have taken.

Change three: FR-11 EscalationBundle gains a draft_blocked case. v1 assumed escalation happened without a draft. Once the router settled at the end of the pipeline, every ticket had a draft, even the ones the guardrails blocked. v2 says a blocked draft still travels with the bundle, flagged so the reviewer sees what was caught and cannot send it as-is. Trigger: Daniel's follow-up comment during the mid-build walk-through. *"show me what got caught, not just that it did."*

Change four: R-08 gets a kill switch of its own. v1 named the `CONFIDENCE_THRESHOLD=1.01` configuration as the mechanism. That mechanism needed a restart, shared a constant with threshold tuning and read in the log as a misconfigured threshold rather than a deliberate halt. v2 replaces it with a dedicated `KILL_SWITCH` flag read on every routing call as router rule 0 (D-15), tested (`test_kill_switch_forces_escalate`). Trigger: the Stage 6 checklist question *"how is it tested?"* which the old mechanism could not honestly answer.

Assumptions from v1 that turned out to be wrong

The retrieval score can serve as an answerability signal. It cannot. D-02a and D-10 both dug at this. At the 0.25 cosine floor only 14 of 143 non-answerable dev tickets return nothing, so a non-empty passage list is not evidence the ticket can be answered. The dev-chosen answerability gate reached AUC 0.64 across six features and would have held zero of the local run's fourteen wrong sends. Deciding answerability is the router's job, classifier confidence, the D-07 list, and the guardrails, not the retriever's.

The LLM judge could serve as an evaluation oracle. It cannot, on this task. B-18 calibrated the judge on 50 drafts against one human assessor; pooled Spearman was 0.35 against a 0.70 threshold. The judge is preserved behind the harness's `-j judge` flag, off by default, and its scores are excluded from every reported figure. This is why citation accuracy and hallucination rate are reported as "not measured to method" in Section 7.

Free-tier model choice would be stable across three weeks. It was not. The Setup Guide's default (`llama-3.1-8b-instruct` on OpenRouter) moved to paid pricing in week two. D-01a records the diagnostic runs on paid providers that carried the project through week three and D-08's response cache plus the local Ollama stack (`llama3.1-8b-ctx8k` for drafts) that closed the free-tier gate on 19 September.

What I decided not to change

The confidence threshold at 0.85. B-16's sweep across 0.50–0.95 showed no threshold reaches the 0.95 precision floor Marcus asked for (ceiling 0.760). 17 of 18 false positives are tickets the docs cannot answer, so answerability, not confidence, is the binding constraint. Lowering the threshold gains a little coverage at real cost to precision; raising it holds correctly-answerable tickets. The revision log records this as "provisional, precision floor unmet" rather than as a solved problem, and the feedback loop is what would let it be resolved.

The persistent `decisions.db` file. Wiping it per run would remove the `run_id`-scoping code and the class of bug it exists to prevent. It stays because the persistent file is useful for the demo and A8 reconciliation is trivially satisfied by `WHERE run_id = ?`.

The reflection

What I most misunderstood in v1. The routing decision as a branch in the middle of the pipeline, with generation and guardrails as branches off `auto_respond`. The code never worked that way and the docs finally caught up on 20 September. Routing has to run last because two of its rules read verdicts from stages that come after any earlier place in the flow. This is not a wording change; it is why the `EscalationBundle` in FR-11 can carry a blocked draft.

Which discovery work would have caught it earlier. A walk-through of an escalated ticket with Daniel on day two, before any code was written. The concrete request *"give me what you already searched, what you thought it was, and any draft"* implies routing after generation. It was there in the interview; I read it as a data-shape request rather than a control-flow one.

What I would do differently on Monday. Build the fairness audit first, before the harness. Every run would report the language-fluency gap from the beginning, and the query-rewriting pass would have been in v1 of the retrieval component rather than the second item in the "what I would do next" list.

What is still uncertain. The precision floor. Marcus asked for 0.95; the best the confidence threshold delivers is 0.76. This is a real gap. Closing it needs either a query-rewriting retrieval pass that lifts answerability, or an answerability model that beats AUC 0.64. Both are three-week work of their own and neither is guaranteed to hit 0.95.

SECTION 10

Conclusions

What was built and what it achieved

A six-component RAG pipeline that reads a support ticket, retrieves the documentation, drafts a reply, checks it against six safety guardrails, and either sends it, blocks it, or hands it to a human with the working attached. It runs from a clean checkout under one command, processes 80 tickets end-to-end unattended with zero manual intervention, and produces a metrics report and a reconciling decision log automatically. The 20 September graded run at 80/80 is the evidence, no PII in outbound replies, retrieval hit@3 at 98.1 per cent, intent accuracy 88.8 per cent, kill switch tested and CI-gated, wall clock 6.8 minutes at \$0.80. The 19 September free-tier local run is the corroborating evidence that the free-model gate the Build Spec rewards is met.

It moves work off the queue at a directional improvement over the 58 per cent baseline escalation rate. What it does not yet do is resolve *more* tickets correctly than the humans were, because the send-precision problem, 13 wrong sends of 48, is bounded by the confidence-threshold precision floor being unreachable at 0.95 on this stack. The report has been honest about this.

What I would do next, in order of expected value

1. Latency, close the last factor of two. On the graded paid stack the guardrail stage is p50 2.3 s and the generator 1.8 s, so the ticket p95 is 6.7 s, a miss against the 3 s target by a factor of two, not the twenty-times of the earlier free-tier measurement. Two levers remain. Running guardrails against a partial generator response instead of waiting for the whole one is the structural win, the guardrails do not need the last two sentences of a reply to check for PII or grounding, and starting them earlier removes a serial waypoint. Second lever: a first-pass extractive answer for the fraction of tickets where the corpus supplies a direct quote (which is most of them), letting the full generator run only when extraction fails. Either reaches the 3 s target on this stack; a smaller model does not.

2. Fairness, query rewriting at retrieval time. The language-fluency gap has narrowed from 44 pp to 20 pp between the 15 and 20 September runs, on a sample too small (n=19) to call the improvement significant. It still misses the 5 pp target. The narrower fix is a query-rewriting pass at retrieval time that normalises phrasing towards the vocabulary the docs use, before any deeper embedding-space swap. One-week piece of work with a measurable output (repeat the audit); addresses the Governance Framework's warned-about disparity directly.

3. Feedback loop, build the table. One new table in `storage/decisions.db` (schema shape already in `architecture.md`), one write path from the reviewer's UI, one report under `evaluation/`. Five verdict values. `sent_as_drafted`, `edited_then_sent`, `rewritten`, `wrong_article`, `no_article_exists`. This is what Ines's PRD success criterion asks for. *"every response the writer flags as 'article wrong' vs 'system misread' can be routed to the correct fix"*, and nothing in the build satisfies it today. Explicit non-goal: no online learning. Feedback collects data for an offline decision by a person; it must not retune a threshold automatically, or two runs over the same input would disagree (breaking FR-09 and A5).

What remains uncertain

Whether a 0.95 precision floor is achievable on this corpus at all. Marcus asked for it; B-16 could not find it. The three routes to close the gap, query rewriting, a better answerability model, a stronger classifier, each

have a plausible ceiling and none has been shown to hit 0.95 in isolation. The honest position is that this system delivers a directional improvement with tight safety properties, and that the next set of experiments is what tells us whether 0.95 is available or whether the target itself needs a conversation with Marcus about what precision buys him.

Whether the hidden evaluation set will move the numbers. It might. The label audit describes contested-text tickets that inflate the wrong-decision count; the hidden set carries whatever labels it carries and could have more, fewer or none of them. 21 of the 80 validation tickets carry contested text and 10 of the 26 wrong decisions on the 20 September graded run land on them. The system's behaviour is not tuned to the validation set. `grep -r VAL- src/` returns no rule keyed on a ticket ID, but the threshold was swept on it, and that is the exposure.

What I would tell CloudServe on Monday morning

That the system does what they need at the shape they need it: answers what it can defend, holds what it cannot, and lets a reviewer see the working every time. That the headline numbers show it moving work off the queue while the correctness of what it sends is at the human baseline, not above it, so the queue gets shorter and the customer experience does not get better yet. That the next version should ship with the feedback loop wired, so the reviewer's verdicts become the measurement everything else is tuned against. And that the latency figure on production providers is 6.7 s p95 today, twice the target rather than twenty times, and closable without leaving the current stack.

APPENDIX A

Evaluation results table

Full measure table from `evaluation/results/run_20260920/results_table.md`. Run harness-20260920T173736Z-8eee28c5 on 80 validation tickets. Code a288388. Drafting model gpt-4o-mini, guardrail model gpt-4o, confidence threshold 0.85, guardrails on. Model cache on (31 cached calls, 273 live; 9 tickets partly replayed). Wall clock 406.7 s. Cost \$0.80 (saved by cache \$0.09). Runs against the hidden evaluation set: 0.

Measure	Baseline	Target	Achieved	Confidence
First-contact resolution	42 %	60 %	43.8 % correctly resolved (60.0 % auto-sent)	95 % CI 33–55 %
Mean time to first reply	8–12 hrs	< 5 min	5.3 s (median 5.8 s)	n=48
Satisfaction proxy	3.2 / 5	4.0	Not measured to method	,
Escalation rate	58 %	≤ 30 %	40.0 % (24 routed, 8 blocked)	95 % CI 30–51 %
Classification precision	,	85 %	82.0 % mean over 22 classes; 15 of 22 ≥ 85 %	Median n per class 3
Hallucination rate	,	≤ 5 %	Not measured to method	,
Citation accuracy	,	95 %	Not measured to method; article match 95.3 %	n=53
Latency p95	,	< 3 s	6.7 s (max 7.0 s)	n=71 live
Private data occurrences	,	0	0	48 sent scanned
Cross-group variation	,	< 5 pp	20 pp (language_fluency non-fluent vs fluent)	n=19, not sig.

Table A.1 — Full evaluation results, verbatim from the run's results_table.md.

Governance conditions

Condition	Requirement	Result	Status
Private data in outbound text	Zero occurrences	0 (automated scan of all 48 sent replies)	MET
Quality across customer groups	< 5 pp variation	20 pp (language_fluency non-fluent vs fluent, n=19; not sig.)	NOT MET
Decision logging	Complete coverage	80 of 80 tickets logged; reconciles = True	MET
Confidence calibration	Stated ≈ observed within 5 pp	Largest gap –35 pp in the 0.7–0.8 band (n=10)	NOT MET

Table A.2 — Tier-three governance conditions.

Routing outcomes against the labels

Measure	Value
Replies sent	48 of 80
Send precision (sent replies that should have been sent)	72.9 % (35 of 48)
Send coverage (tickets that should be answered, answered)	72.9 % (35 of 48)
Wrong sends	13 (16.2 % of tickets)
Wrong holds	13 (16.2 % of tickets)

Table A.3 — Where the send/hold decisions were right and where they were wrong.

Stage of wrong decisions

Stage : reason	Tickets
answerability : sent_unanswerable	12
classification : low_confidence	9
guardrail : answer_relevance : verdict	3
guardrail : grounding : verdict	1
other : sent_against_label	1

Table A.4 — Each wrong decision attributed to the earliest stage that explains it.

Where the time goes (live tickets, n=71)

Stage	p50 (s)	p95 (s)	mean (s)
classification	1.451	1.786	1.475
retrieval	0.116	0.316	0.130
generation	1.799	2.534	1.787
guardrails	2.331	3.105	2.149

Table A.5 — Stage latency on live tickets. Model calls per ticket mean 3.77, max 4.

APPENDIX B

Fairness audit

Full segment table from the 15 September paid diagnostic run (b21_openai_gemini_80_20260915/fairness/fairness_table.md), the earlier run this audit reports against, since the 19 September local run does not carry a fairness sub-directory. Quality score is decision correctness against the labels; the gap column is percentage points from the best-performing segment in the group. Segments under n=10 are reported for completeness and marked indicative.

Segment	n	Resolution	Quality	Gap (pp)	Note
language_fluency = fluent	61	75 %	72 %	+0	40 auto-send, 17 unanswerable; mean conf 0.82
language_fluency = non_fluent	19	62 %	47 %	+25	8 auto-send, 10 unanswerable; mean conf 0.83
customer_region = asia_pacific	21	67 %	67 %	+1	15 auto-send, 4 unanswerable; mean conf 0.85
customer_region = europe	25	75 %	68 %	+0	16 auto-send, 6 unanswerable; mean conf 0.84
customer_region = latin_america	7	67 %	71 %	-3	n<10 indicative
customer_region = north_america	27	79 %	63 %	+5	14 auto-send, 13 unanswerable; mean conf 0.81
customer_tier = business	30	62 %	50 %	+29	21 auto-send, 8 unanswerable; mean conf 0.82
customer_tier = enterprise	8	67 %	62 %	+16	n<10 indicative
customer_tier = standard	42	86 %	79 %	+0	21 auto-send, 17 unanswerable; mean conf 0.81
channel = chat	22	69 %	73 %	+0	13 auto-send, 9 unanswerable; mean conf 0.79
channel = docs_comment	16	50 %	56 %	+16	8 auto-send, 6 unanswerable; mean conf 0.77
channel = email	31	81 %	65 %	+8	21 auto-send, 9 unanswerable; mean conf 0.86
channel = forum	11	83 %	73 %	+0	6 auto-send, 3 unanswerable; mean conf 0.82
length = long	24	80 %	62 %	+12	15 auto-send, 6 unanswerable
length = medium	28	78 %	75 %	+0	18 auto-send, 8 unanswerable
length = short	28	60 %	61 %	+14	15 auto-send, 13 unanswerable

Table B.1 — Fairness audit, full segment table.

APPENDIX C

Decision index (D-01 through D-16)

One line per ADR. Full text under docs/adr/D-XX-*.md.

ID	Decision
D-01	Model provider = OpenRouter (Llama 3.1 8B). Setup Guide default, single API key.
D-01a	After the free model failed: paid diagnostics reported honestly; free model still OPEN, closed by D-01b's 19 Sep local run.
D-02	Search-embedding model = all-MiniLM-L6-v2, verified on 357 dev tickets; better on non-fluent English.
D-02a	Retrieval distance = cosine, threshold 0.25, set by measurement; the l2 default discarded 42 % of correct answers.
D-03	Decision log = SQLite at storage/decisions.db, standard library only.
D-03a	Decision log trace schema, one row per stage, keyed by ticket_id and run_id.
D-03b	Log-write failure = swallow, flag, escalate, degrades the action to preserve the record.
D-04	Chunking = 800/120, title + category prepended, B-30 lifted dense hit@3 from 91.0 to 95.2 %.
D-05	Confidence threshold = data-driven sweep (provisional 0.80 until B-16).
D-05a	Confidence threshold's dual role, routing gate + reported score, deliberately not split.
D-05b	Measured confidence threshold = 0.85 (provisional, precision floor 0.95 UNMET).
D-06	Agent pattern = Router with inline Self-RAG structural critic; retry cap 1.
D-06a	Self-RAG critic = structural in loop; grounding as a blocking guardrail (as built).
D-07	Never-auto-respond = fixed intent policy list of 4 categories + unknown.
D-08	Model response cache. Build Spec calls rate limits a design problem.
D-09	Retrieval query text = subject + body, dense only. rRF hybrid earned nothing here.
D-10	No answerability gate from retrieval, six features reach AUC 0.64, gate holds 0 of 14 wrong sends.
D-11	PII LLM detections = value-shaped only, drops the three false-positive blocks from the local run.
D-12	Guardrails concurrent, max 4 workers, identical verdicts, guardrail stage 10.1 → 3.1 s on paid.
D-13	Bounded model call, caller-side deadline (MODEL_CALL_DEADLINE_SECONDS=180) prevents hangs breaking A9.
D-14	Confidence-floor failure = escalate at rule 6 (not block at rule 2), 17 of 33 blocks were floor-only.
D-15	Kill switch = KILL_SWITCH env, read every routing call as router rule 0, tested.
D-16	Injection detection = input-side heuristic, escalate + record (router rule 2b), six-attack probe passed.

Table C.1 — Decision index.

APPENDIX D

Declaration and AI-tool use

Per the Submission Guide's final checklist item. *"the report contains your declaration of AI tool use."*

Declaration

All code, design decisions, and evaluation figures in this submission are my own work, produced under the FDE Capstone brief between 26 August 2026 and 20 September 2026. Every requirement, ADR, prompt, test, and result is traceable to the evidence line it cites; the audit-time chain (evidence tag → requirement → code → test) is preserved in the repository. No credentials, keys, or tokens appear anywhere in the source or the git history (checked by `tests/test_config_keys.py` as a CI gate).

The dataset is CloudServe's supplied fixture, 500 development tickets, 80 validation tickets, 29 documentation articles, 200 reference responses, five stakeholder interview transcripts. The hidden evaluation set was not accessed. Runs against the hidden set: **zero**.

AI tools used

Tool	Where used	How it was used
Claude (Sonnet, Opus)	Draft prompts (PR-*), draft ADRs, draft prose for docs, exploration of design options.	As a pair-programming interlocutor; every accepted output was reviewed line-by-line and edited. No output was accepted verbatim without change.
Cursor / Copilot-style completions	src/ during weeks 2–3.	Line-level completions inside my own draft, comments, docstrings, boilerplate. No component was scaffolded end-to-end from a prompt.
Model providers under test	The system itself.	OpenRouter (Llama 3.1 8B, discontinued mid-project, see D-01a), then Groq / OpenAI / Gemini as paid diagnostics, then local Ollama (llama3.1-8b-ctx8k for drafting, qwen2.5-7b-ctx8k for guardrails) as the graded configuration.

Table D.1 — AI tool use across the project.

Attribution of non-original code

Third-party dependencies are pinned in `requirements.txt` and appear in the attribution section of the README. The evaluation harness, the six pipeline components, the guardrail set, the decision log, the metrics writer, and every ADR were written for this project. The FastAPI wrapper (`src/api.py`) uses the standard FastAPI + Pydantic patterns; no code was copied from another codebase.

Contact

Alok Kulkarni. Submission dated 20 September 2026. The FDE Capstone Stage 6 Submission Guide is the authoritative reference for what this report contains and where it sits in the submission archive.